

Smali By bithowl: Chapter 10 Register Types

("What's Actually Inside the Register?")



SMALI BY BITHOWL : CHAPTER 10

REGISTER TYPES

"What's Actually Inside the Register?"

```
const/4 v0, 0x1
const-string v1, "admin"
new-instance v2, Lcom/example/User;
invoke-virtual {v2}, Ljava/lang/Object;.>...
return v0
.end method
```



THE STORY (HOOK)

You discover a locked room inside an Android app.

 **Safe**
Stores information

 **Control Panel**
Performs actions

 **Telephone**
Communicates outside

 **Filing Cabinet**
Stores records

In programming:

Fields → Stores

- username
- token
- API_URL
- isLoggedIn

Methods → Does

- login()
- logout()
- verifyToken()
- sendRequest()

These two pieces reveal the purpose of a class before you read its code.

REGISTER TYPES – THE BIG PICTURE

REGISTER VALUES

Single-Width (1 register)

- int I
- float F
- reference (object / array / string)
- Object L...

Wide (2 registers)

- long J
- double D

Single-width → 1 register | Wide → 2 consecutive registers

WHY ANDROID USES REGISTERS?

- ✓ Dalvik/DEX uses register-based architecture (not stack-based like JVM).
- ✓ Instructions operate directly on registers.
- ✓ Smaller bytecode, faster execution.
- ✓ Every value lives in a register.

```
const/4 v0, 0x5
const/4 v1, 0xa
add-int v2, v0, v1
```

After execution

v0	5
v1	10
v2	15

v0 = 5, v1 = 10, v2 = v0 + v1 = 15

THE TWO REGISTER FAMILIES

v Registers (Local / Working)
Used for local or temporary values.
v0 v1 v2 v3 ...

p Registers (Parameter Registers)
Used for method parameters.
p0 p1 p2 p3 ...

v = local / working registers | p = parameter registers

Not two separate sets – just different names within the same register frame.

INSTANCE METHODS

Belongs to an object.
Operates on a specific instance.

```
.method public verify(Ljava/lang/String;)Z
...
.end method
```

Call using: invoke-virtual / invoke-direct

```
invoke-virtual {p0, p1}, Lcom/example/AuthManager;.>verify(Ljava/lang/String;)Z
```

Example: manager.verify(token);

STATIC vs INSTANCE METHODS

Type	Belongs to	Common Invocation
Static method	Class	invoke-static
Instance method	Object	invoke-virtual
Private / direct method	Specific object / class context	invoke-direct

Example:

Static
MathUtils.calculate();
(no object needed)

Instance
user.getToken();
(operates on object)

CONSTRUCTORS

Special methods that initialize objects.
Default constructor:

```
.method public constructor <init>()V
...
.end method
```

With parameter:

```
.method public constructor <init>(Ljava/lang/String;)V
...
.end method
```

Return type is V (void).

CLASS INITIALIZER <clinit>

Initializes static fields.
Runs when class is first loaded.

```
.method static constructor <clinit>()V
...
.end method
```

Example Java:

```
static {
    API_URL = "https://api.example.com";
}
```

Great place to find default config, endpoints, feature flags, secrets.

METHOD SIGNATURES

Signature = name + parameters + return type

Small Signature	Decodes To (Java)
login()V	void login()
login(Ljava/lang/String;)Z	boolean login(String)
login(Ljava/lang/String;Ljava/lang/String;)Z	boolean login(String, String)
getToken(Ljava/lang/String;)Z	String getToken()
calculate(II)I	int calculate(int, int)

From: methodName → (Param1)Param2... → ReturnType

PARAMETERS & RETURN VALUES

Parameter registers: p0 p1 p2 ...

Return instructions:

- return-void → V (void)
- return-v0 → Primitive (int, boolean, etc.)
- return-object v0 → Object / reference

p0 is this for instance methods.

WHAT IS A FIELD?

Data associated with a class or object.
Java: private String token;

Small: .field private token:Ljava/lang/String;

Access (private) → Name (token) → Type (String)

Fields can store anything: primitives, objects, arrays, collections, references, etc.

STATIC FIELDS

Belongs to the class (one shared value).

```
.field public static API_URL:Ljava/lang/String;
```

Common uses:

- Constants
- Global state
- Configuration
- Singletons
- API endpoints
- Feature flags

Example: .field private static API_KEY:Ljava/lang/String;

A suspicious field name is NOT automatically a vulnerability. Investigate how it's used!

INSTANCE FIELDS

Belongs to each object (separate values).

```
.field private username:Ljava/lang/String;
.field private token:Ljava/lang/String;
.field private loggedIn:Z
```

User A: username = "alice", token = "...", loggedIn = true
User B: username = "bob", token = "...", loggedIn = false

Each object has its own copy of instance fields.

ACCESSING FIELDS (INSTRUCTIONS)

Instance fields (need object)
Read: lget / lget-object
Write: sput / sput-object
Example (read): lget-object v0, p0, Lcom/example/User;.>token:Ljava/lang/String;
Conceptually: v0 = this.token
lget/lput → instance

Static fields (no object)
Read: sget / sget-object
Write: sput / sput-object
Example (read): sget-object v0, Lcom/example/Config;.>API_URL:Ljava/lang/String;
Conceptually: v0 = Config.API_URL
sget/sput → static

REGISTER TYPE REFERENCE

Java Type	Small Descriptor	Width	Example Value
boolean	Z	1	v0 = true / false
byte	B	1	v0 = 0x7F
char	C	1	v0 = 'A'
short	S	1	v0 = 1234
int	I	1	v0 = 42
float	F	1	v0 = 3.14f
long	J	2	v0v1 = 123456789L
double	D	2	v0v1 = 3.14159
String (object)	Ljava/lang/String;	1 (ref)	v0 → "hello"
Object (any)	Lpackage/Class;	1 (ref)	v0 → User object
int[] (array)	[I	1 (ref)	v0 → {1, 2, 3}
String[] (array)	[Ljava/lang/String;	1 (ref)	v0 → {"a", "b"}
void	V	---	return-void

REGISTERS ARE REUSABLE

A register can hold different values at different times.

```
const/4 v0, 0x1 # v0 = int
...
const-string v0, "hello" # v0 = String reference
...
lget-object v0, p0, Lcom/example/User;.>username:Ljava/lang/String; # v0 = Object reference
```

Same register, different value, different type. Type is determined by instructions and usage.

SOME VALUES USE MULTIPLE REGISTERS

Wide values need two consecutive registers.

Long (J)	v0	v1	→ one long value
Double (D)	v2	v3	→ one double value

Most values → 1 register (long, double) → 2 registers

Important when understand registers!

BUG HUNTER PERSPECTIVE

Follow the data flow through registers.

```
Input (p1) → v0 (value) → Process (method call) → v1 (result) → Decision / values
```

Ask these questions:

- Where did this value come from?
- Where is it stored?
- Where is it transformed?
- Where is it passed?
- Where is the result used?
- Is it validated?
- Can attacker influence it?
- Does it lead to a privileged action?

PRACTICAL EXERCISE – REGISTER DETECTIVE

- Extract APK: apktool d app.apk
- Pick a method: ... multiple registers.
- Map every register (p0, p1, ..., v0, v1, ...).
- Determine the type of each register:
 - Method descriptors → const / move instructions
 - move-result / move-result-object → Field descriptors
 - Invocation signatures → Array instructions
- Trace the value flow from start to return.

MINI CHALLENGE (Decode it!)

```
.method public check(Ljava/lang/String;)Z
    .locals 2
    move-result-object p1, Ljava/lang/String;.>length()I
    Decode:
    const/4 v1, 0x5
    if-eq v1, v1, const_0
    const/4 v0, 0x0
    return v0
    .const_0
    const/4 v0, 0x1
    return v0
    .end method
```

Conceptual Java: return input.length() == 5;

QUICK RECAP

- ✓ Registers hold all values used in a method.
- ✓ Single-width → 1 register
- ✓ Wide (long/double) → 2 registers.
- ✓ Object, String, Array → references.
- ✓ Type depends on usage, not on name.
- ✓ p0 is "this" in instance methods.
- ✓ locals = local registers, registers = total.
- ✓ Follow data flow to understand behavior.

FINAL THOUGHT

Registers are not just boxes. They carry data, intent, and consequences.

Where it comes from? → What it becomes? → Where it goes?

By – bithowl



In the last chapter, we learned about:

v0

v1

v2

p0

p1

We learned that registers are the tiny storage locations used by Smali methods. But there's a question we haven't answered yet:

What can actually be stored inside those registers?

The answer is: Almost everything a method needs.

Numbers.

Booleans.

Objects.

Strings.

Arrays.

And even values that require **two registers**. That's where register types become important.

Because when you're reading Smali, you shouldn't just ask: "What's in v0?"

You should ask: "What type of value is v0 holding?"

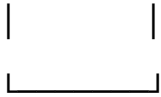
That one question can completely change how you interpret the next instruction.

Let's break it down.

The Story (Hook)

Imagine a warehouse with identical storage boxes. Every box looks exactly the same from the outside.





But inside, one box contains: 5

Another contains: "admin"

Another contains: User object

Another contains: [10, 20, 30]

And one particularly large item needs **two boxes**. The boxes themselves don't tell you what they're holding. You need to understand how the program is using them.

That's essentially what happens with registers.

Register



Value



Type determines how the value is interpreted

So:

v0 → int

v1 → String reference

v2 → array reference

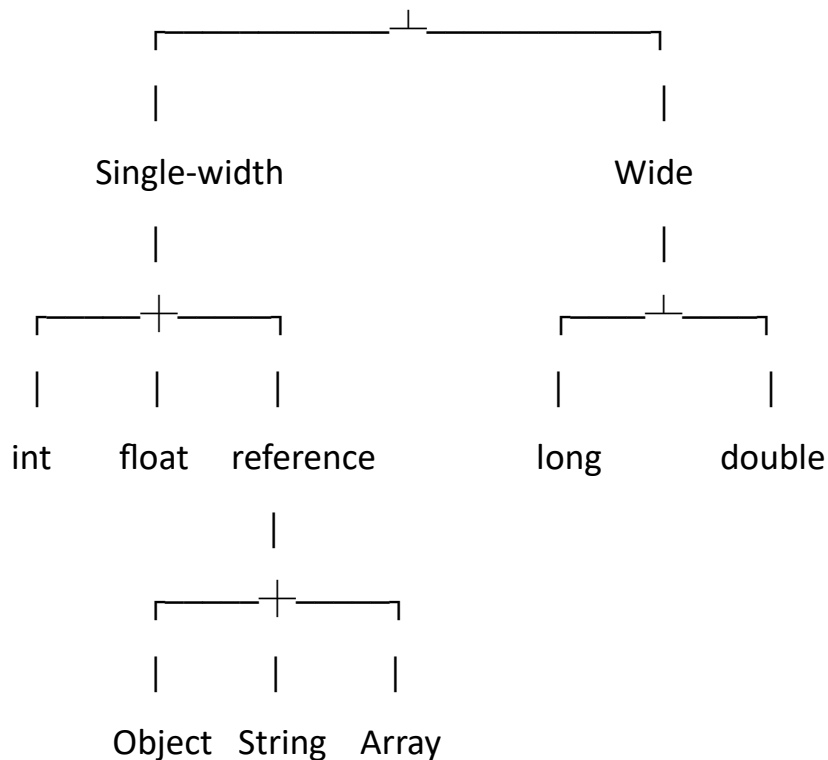
The register name doesn't tell you the type. The **instructions and surrounding code do**.

Register Types - The Big Picture

For beginners, divide register values into these categories:

REGISTER VALUES





A useful simplified rule is:

Single-width values - Usually occupy **one register**. Examples: int, float, Boolean, byte, char, short, object references, array references, String references

Wide values - Occupy **two consecutive registers**. Examples: long, double

This distinction becomes extremely important when reading register layouts.

1. Single-Width Registers

A single-width value occupies one register.

For example: `const/4 v0, 0x5`

Conceptually: `v0 = 5`

Only one register is required.

Another example: `const/4 v1, 0x1`

Conceptually: `v1 = 1`

So:

v0 → 5

v1 → 1

These are single-width values.

1.1 Integer Values

Integers are among the most common values you'll encounter. Smali represents the Java int type using: I

Example method: .method public calculate()I

The I at the end means: Return type = int

An integer can be stored in a single register.

Example: const/4 v0, 0x5

Conceptually: v0 = 5

You might then see: add-int v2, v0, v1

Conceptually: v2 = v0 + v1

For a bug hunter, integer registers commonly appear in:

- Counters
- IDs
- Flags
- Status codes
- Array indexes
- Length calculations
- Permission checks

1.2 Boolean Values

A boolean also fits in a single register. Smali uses: Z for the Java boolean type.

For example: .method public isAuthenticated()Z

means: Returns boolean

You may encounter:

const/4 v0, 0x1

return v0

Conceptually:

return true;

And:

const/4 v0, 0x0

return v0

conceptually means:

return false;

This pattern is particularly interesting when analysing authentication or authorization logic.

1.3 Float Values

The Java float type uses: F and occupies one register.

Example: const/high16 v0, 0x3f800000

The exact encoding can look unfamiliar at first, but the important concept is:

float → single-width value

Floating-point operations have their own Smali instructions, such as:

add-float

sub-float

mul-float

div-float

You don't need to memorize every instruction yet.

The important thing is recognizing the type.

2. Wide Values

Now we reach something different. Some values require **two consecutive registers**. These are called **wide values**.

The two main examples are:

long

double

So instead of:

$v0 \rightarrow \text{value}$

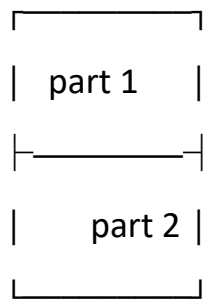
you may have:

$v0 + v1 \rightarrow \text{one long value}$

or:

$v2 + v3 \rightarrow \text{one double value}$

Think of it like storing a large object across two boxes:



one value

This matters enormously when counting registers.

2.1 Long

Java's long type is represented by: J and uses two register slots.

For example: `.method public getTimestamp()J`

The J means: Return type = long

Conceptually, you might encounter a pair such as: $v0 + v1$ representing one long value.

Don't interpret this as:

$v0 = \text{first number}$

$v1 = \text{second number}$

They're together representing **one wide value**.

2.2 Double

Java's double type is represented by: D and also occupies two registers.

For example: `.method public calculate()D`

means the method returns a double.

Conceptually: $v0 + v1 \rightarrow \text{double}$

Again, the two registers together represent one value.

Why Wide Registers Matter

Imagine: `.method public calculate(J)V`

The method accepts one long.

A beginner might think: One parameter = one register

But that's not true here. A long occupies two register slots.

So parameter layout can become: $p0 + p1 \rightarrow \text{long}$ for a static method.

For an instance method, the implicit this reference comes first, so the layout conceptually becomes:

$p0 \rightarrow \text{this}$

$p1 + p2 \rightarrow \text{long}$

This is one reason register counting can become tricky.

Always consider the type descriptor.

3. References

3.1 Object References

Now we leave primitive values. Smali registers can also hold **references to objects**.

For example: `const-string v0, "hello"`

places a String reference into v0.

You might then see:

`invoke-virtual {v0}, Ljava/lang/String;->length()I`

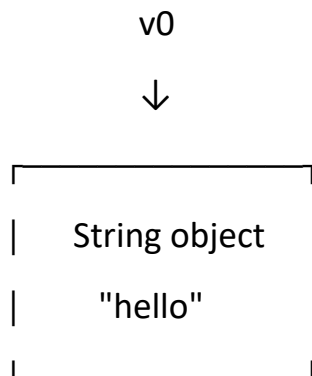
Conceptually:

`v0.length();`

The important distinction is: Register does not contain the entire object.

Register contains a reference to the object.

Think of it like an address card:



This is a fundamental concept.

Object Descriptors

From Chapter 6, you learned that object descriptors look like:

Lpackage/ClassName;

For example: Ljava/lang/String;

represents: java.lang.String

And: Lcom/example/User;

represents: com.example.User

So if you see:

iget-object v0, p0, Lcom/example/User;->username:Ljava/lang/String;

you can understand:

Read an object field



Store its reference in v0

3.2 Strings

Strings are objects in Java. That means a String is not a primitive type.

The descriptor is: Ljava/lang/String;

For example: `const-string v0, "admin"`

Conceptually: `String value = "admin";`

Now: `v0` → String reference

This distinction is important.

Don't think: `v0` = raw string bytes

Instead: `v0` → reference to String object

Why Strings Matter to Bug Hunters

Strings are everywhere in Android applications.

You may find: URLs, API endpoints, Feature flags, Error messages, File paths, Intent actions, Database names, Configuration values, Potential secrets

For example:

```
const-string v0, "https://api.example.com"
```

or:

```
const-string v1, "debug_mode"
```

or:

```
const-string v2, "Authorization"
```

These strings can provide excellent reconnaissance clues.

But again:

Finding a string isn't automatically finding a vulnerability.

You need to understand how the application uses it.

3.3 Arrays

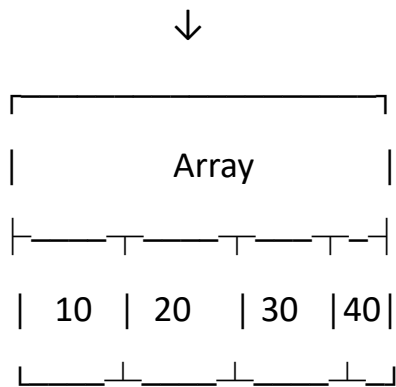
Arrays are also represented by references.

For example: `int[] values;`

The register contains a reference to the array.

Conceptually:

`v0`



The register itself isn't storing every element. It holds a reference to the array object.

Array Descriptors

Array descriptors begin with: [

Examples: [I

means: int[]

And: [Ljava/lang/String;

means: String[]

For example: .method public getUsers()[Ljava/lang/String;

means:

Method:

getUsers()

Returns:

String[]

Reading Array Operations

You may encounter instructions such as: aget, aput

These are commonly used to read from and write to arrays.

Conceptually: aget v0, v1, v2

means: v0 = array[index]

where:

v1 → array

v2 → index

Similarly: aput v0, v1, v2

conceptually means: array[index] = value

The exact instruction variant depends on the element type.

Object vs Primitive — The Important Difference

Consider these two:

|

Ljava/lang/String;

They represent very different things.

|

↓

int

↓

Primitive value

Whereas:

Ljava/lang/String;

↓

String

↓

Object reference

This difference affects the instructions used.

For example: add-int works with integers.

While: invoke-virtual may operate on an object reference.

Similarly: return is commonly used for primitive values.

While: return-object is used for object references.

Recognizing these patterns makes Smali much easier to decode.

A Register Can Change What It Holds

Here's a subtle but important concept. A register isn't permanently assigned one type for the entire method.

For example: `const/4 v0, 0x1`

At this point: `v0` → integer value

Later: `const-string v0, "hello"`

Now: `v0` → String reference

The same register has been reused.

So don't say: "`v0` is an integer register."

Instead say: "At this point in execution, `v0` contains an integer."

Later, it may contain something completely different.

Type Is Determined by Usage

Suppose you see: `move-object v0, p1`

The object suffix tells you that the operation is moving an object reference.

Compare: `move v0, p1`

and: `move-object v0, p1`

The instruction itself gives you valuable information about how the register is being interpreted.

Similarly:

`move`

`move-object`

`move-wide`

represent different categories of values.

This is why understanding instructions and register types together is so powerful.

Method Calls Reveal Types

Consider:

```
invoke-virtual {v0}, Ljava/lang/String;->length()I
```

```
move-result v1
```

What do we know?

The method being called is:

```
String.length()
```

and it returns: I

Therefore: v1 → int

Now consider:

```
invoke-virtual {v0}, Lcom/example/User;->getName()Ljava/lang/String;
```

```
move-result-object v1
```

The return descriptor is:

```
Ljava/lang/String;
```

Therefore: v1 → String reference

This is one of the easiest ways to infer register types while reverse engineering.

A Complete Example

Consider:

```
.method public getUserInfo(Ljava/lang/String;)Ljava/lang/String;
```

```
.locals 3
```

```
const-string v0, "Hello"
```

```
invoke-virtual {p1}, Ljava/lang/String;->length()I
```

move-result v1

invoke-static {v1}, Ljava/lang/Integer;->toString()Ljava/lang/String;

move-result-object v2

return-object v2

.end method

Let's trace the registers.

Parameter

p0 → this

p1 → String

First instruction

const-string v0, "Hello"

Therefore: v0 → String reference

Method call

invoke-virtual {p1}, Ljava/lang/String;->length()I

The String length is calculated.

Result

move-result v1

Therefore:

v1 → int

Next call

invoke-static {v1}, Ljava/lang/Integer;->toString()Ljava/lang/String;

The integer is converted into a String.

Result

move-result-object v2

Therefore:

v2 → String reference

Finally:

return-object v2

The method returns the String reference.

The flow becomes:

p1

|



String

|



length()

|



v1

|

int

|



Integer.toString()

|



v2

|
String

|
▼

return

That's register-type tracing.

Bug Hunter Perspective

This isn't just theory.

When hunting Android vulnerabilities, knowing register types helps you understand what data is moving through security-sensitive code.

Imagine:

p1 → user-controlled String

↓

v0

↓

validation()

↓

v1

↓

authorization check

Now you can ask: What type is p1?

Where did it come from?

Was it validated?

What does v1 represent?

Does the result control access?

Or:

v0 → Content URI



ContentResolver



database query

Now you've identified a potential data-flow path.

Or:

v0 → String URL



WebView.loadUrl()

Now you can investigate how that URL is constructed and whether it is attacker-controlled.

The register type gives you the first piece of the puzzle.

Bug Hunter Pattern — Primitive to Security Decision

Consider:

```
invoke-virtual {p1}, Lcom/example/Auth;->isAdmin(Ljava/lang/String;)Z
```

```
move-result v0
```

```
if-eqz v0, :cond_0
```

We can decode:

```
p1 → input
```



```
isAdmin()
```



```
boolean
```



```
v0
```



conditional branch

Here v0 isn't just some random register. It's carrying a **security decision**.

That's exactly the kind of data flow a bug hunter should pay attention to.

Bug Hunter Pattern — Object Reference

Consider:

```
iget-object v0, p0, Lcom/example/User;->token:Ljava/lang/String;
```

Now:

p0 → User object

v0 → token String reference

Then:

```
invoke-virtual {v0}, Ljava/lang/String;->length()I
```

```
move-result v1
```

Now:

v1 → int

So the type changes as the value moves through the program:

User object



String



int

This is exactly why type awareness is essential for reverse engineering.

Quick Type Reference

Keep this table nearby while reading Smali:

Java Type	Smali Descriptor	Width
boolean	Z	1

Java Type	Smali Descriptor	Width
byte	B	1
char	C	1
short	S	1
int	I	1
float	F	1
long	J	2
double	D	2
void	V	—
String	Ljava/lang/String;	1 reference
Object	Lpackage/Class;	1 reference
int[]	[I	1 reference
String[]	[Ljava/lang/String;	1 reference

Remember:

Object and array values are references.

They occupy one register in this model, regardless of the size of the object they refer to.

Practical Exercise Become a Register Detective

Extract an APK: `apktool d app.apk`

Pick a small Smali class. Find a method containing several registers.

For every register, create a simple map:

p0 → ?

p1 → ?

v0 → ?

v1 → ?

v2 → ?

Then determine the type from:

- Method descriptors
- const instructions
- move instructions
- move-object
- move-wide
- Field descriptors
- Method return types
- Array instructions
- Invocation signatures

For example:

p0 → this

p1 → String

v0 → int

v1 → String

v2 → boolean

Then follow each value through the method.

Your goal isn't to understand the entire application.

Your goal is to answer:

What type of value is moving through this register right now?

Mini Challenge

Decode this:

```
.method public check(Ljava/lang/String;)Z
```

```
.locals 2
```

```
invoke-virtual {p1}, Ljava/lang/String;.>length()I
```

```
move-result v0
```

```
const/4 v1, 0x5
```

```
if-ge v0, v1, :cond_0
```

```
const/4 v0, 0x0
```

```
return v0
```

```
:cond_0
```

```
const/4 v0, 0x1
```

```
return v0
```

```
.end method
```

Let's reason through it.

Parameter

p0 → this

p1 → String

Method call

p1.length()

returns an:

int

Therefore:

v0 → int

Next value

const/4 v1, 0x5

Therefore:

v1 → int

Comparison

if-ge v0, v1, :cond_0

The code compares two integers.

Return

v0 = 0 → false

or:

v0 = 1 → true

So the conceptual Java is:

```
public boolean check(String input) {  
    if (input.length() >= 5) {  
        return true;  
    }  
  
    return false;  
}
```

You just performed:

Register identification



Type identification



Data flow



Control-flow interpretation



Java reconstruction

That's the foundation of Smali analysis.

One Important Rule

Never identify a register's type just because of its name.

There is no:

v0 = int register

v1 = String register

v2 = object register

Instead:

v0



What instruction wrote to it?



What type did that instruction produce?



How is v0 used afterward?

That's the correct way to reason about registers.

Quick Recap

- Registers are storage locations used by DEX methods.
- Single-width values normally occupy one register.
- long and double are wide values and occupy two register slots.
- Primitive types include int, boolean, float, etc.

- Objects are represented through references.
- Strings are objects and therefore use object references.
- Arrays are also references.
- Register names don't permanently define their types.
- The instruction and surrounding code help determine the current type.
- Method signatures are extremely useful for identifying register types.
- `move-result` commonly receives primitive results.
- `move-result-object` receives object/reference results.
- Type tracing is a fundamental reverse-engineering skill.

Final Thought

When you first learned registers, you saw:

`v0`

`v1`

`v2`

They looked like empty boxes. Now you know they aren't just boxes. They're carrying information.

`v0` → 42

`v1` → "admin"

`v2` → User object

`v3` → `int[]`

`v4 + v5` → long

And the real skill isn't memorizing which register contains what.

It's learning to follow the value.

When you see:

`move-result-object v0`

you should immediately ask:

What object just came back?

When you see: move-result v1

ask: **What primitive value was returned?**

When you see: move-wide v2, v4

ask: **What wide value is moving through these registers?**

Because once you can identify **what is inside the register**, Smali stops being a collection of mysterious v0s and v1s.

It becomes a trail of data.

And for a bug hunter...

following that trail is where the interesting stuff begins.

By - bithowl